

Sprint Report 2

PORTFOLIO TASK 3

Unit code: COS40005

Unit Name: Computing Technology Project A

Submission date: 17/05/2026

ACKNOWLEDGMENT OF COUNTRY

Ved Jay Makhijani

I, Ved Jay Makhijani, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

Minh Hoang Duong

I, Minh Hoang Duong, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

Kumarawadu Dinith Thejana

I, Kumarawadu Dinith Thejana, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

Kaveesha Dulanjana Dharmadasa

I, Kaveesha Dulanjana Dharmadasa, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

Lakshit Bansal

I, Lakshit Bansal, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

CONTRIBUTION SUMMARY (THIS SPRINT)

	Contribution	Initiatives	Communication					Respect
Team Member	Finished tasks in time or on time with an acceptable quality	Self-assigned tasks and proactively found solutions to problems	Attended all team meetings	Attended all supervisor meetings	Attended all client meetings	Replied every time within an acceptable timeframe	Kept the team updated about the status	The student respected others and listened to different opinions
Ved Jay Makhijani	Yes, completed 8 independent Sprint 2 tickets covering task planning, execution, mock simulation, pipeline integration, and comparative evaluation	Yes, took on the most technically complex independent work stream this sprint while teammates focused on vision module and PyBullet setup	Yes	Yes	Yes	Yes, communicated via Teams, responded within 6 hours	Yes, shared action_schema.py interface with teammates	Yes
Minh Hoang Duong	Yes, Proactively creating and covering tasks, refactoring, and fixing the codebases.	Yes, proactively create tasks, designing projects, refactoring, cleaning and maintaining codebase throughout the project.	Yes	Yes	Yes	Yes, github communication with meaningful commit messages, actively reply and send messages within the	Yes, proactively update progress, mentioning	Yes

						microsoft team.		
Lakshit Bansal	Yes , completed Sprint 2 execution backend integration, pipeline testing, and execution module implementation	Yes , independently worked on execution pipeline integration, command dispatching, and MockRobot testing	yes	yes	yes	Yes , active on Teams and GitHub discussions	Yes , shared pipeline integration progress and testing results	Yes
Dinith Thejna	Yes , completed Sprint 2 vision-planner integration, safety validation checks, invalid action detection, and task logging implementation with testing evidence.	Yes , independently worked on scene representation integration, planner object validation, invalid action handling, and JSON task log generation/testing.	Yes	Yes	Yes	Yes , active on Teams	Yes , shared implementation progress, testing outputs, and debugging updates with the team	Yes
Kaveesha Dulanjana Dharmadasa	Yes , completed Sprint 2 task planning, primitive action decomposition, workspace safety validation,	Yes , independently researched and developed planner logic, safety guardrails, structured schemas, and pytest validation for	Yes	Yes	Yes	Yes , active on Teams and project communication	Yes , shared implementation progress, debugging updates, testing outputs, and	Yes

	schema integration, and automated testing with implementation evidence.	task decomposition.					integration status with the team.	
--	---	---------------------	--	--	--	--	-----------------------------------	--

CONTRIBUTION DETAILS

In this section, every member of the team should write a summary of what they have contributed during this sprint. All contributions must be supported by evidence, such as the number of GitHub commits, a screenshot of developed screens, a summary/outcome of research, etc.

Ved Jay Makhijani

During Sprint 2, my contributions spanned the entire pipeline — from LLM evaluation through task planning and execution to full end-to-end system integration. I operated as the technical lead for the planning and evaluation workstreams, completing eight independent tickets across four epics.

Key Technical Deliverables

Multi-Model Comparative Evaluation (EVAL-1, EVAL-2, EVAL-3)

The primary objective of the evaluation workstream was to produce an evidence-based recommendation for LLM backend selection. I developed `comparison_report.py` to run structured evaluations across GPT-4o, Gemini 1.5 Pro, and DeepSeek-Chat, using a test suite of 25 curated instructions distributed across six categories: simple commands, spatial references, multi-step sequences, synonym variation, negation, and compound instructions.

Each test case was assigned a ground truth `ParsedInstruction` object, and four metrics were computed per model: parse success rate, instruction accuracy (exact match against ground truth), field-level accuracy (partial credit per field), and mean response latency. Results were exported to `evaluation_metrics.csv` and `evaluation_results.json` for reproducible analysis.

Key findings: GPT-4o achieved the highest overall instruction accuracy, with particular strength in spatial and multi-step categories. DeepSeek showed competitive accuracy in simple and synonym categories at significantly lower latency, making it a viable cost-efficient alternative for less complex tasks. Gemini exhibited the highest variance across categories, suggesting inconsistency in structured output formatting. These findings directly informed the client's decision on LLM selection for Sprint 3.

Comparative Evaluation Summary — with per-model, per-category results from evaluation_metrics.csv.

evaluation_metrics											
model	category	parse_success_rate	instruction_accuracy	action_accuracy	object_accuracy	destination_accuracy	spatial_accuracy	confidence_calibration	avg_latency_ms	p95_latency_ms	error_rate
DeepSeek (deepseek-chat)	OVERALL	100.0	76.0	100.0	96.0	96.0	84.0	68.0	3805.9	5086.7	0.0
Gemini (gemini-3-flash-preview)	OVERALL	84.0	56.0	76.0	80.0	80.0	72.0	60.0	10475.8	22189.0	16.0
GPT-4o (gpt-4o)	OVERALL	100.0	76.0	100.0	96.0	96.0	84.0	76.0	1149.6	1550.8	0.0
DeepSeek (deepseek-chat)	ambiguous	100.0	33.3	100.0	66.7	66.7	100.0	100.0	2690.8	3599.6	0.0
Gemini (gemini-3-flash-preview)	ambiguous	100.0	33.3	100.0	66.7	66.7	100.0	100.0	4487.8	4867.8	0.0
GPT-4o (gpt-4o)	ambiguous	100.0	33.3	100.0	66.7	66.7	100.0	100.0	637.6	951.5	0.0
DeepSeek (deepseek-chat)	edge_case	100.0	75.0	100.0	100.0	100.0	75.0	75.0	3470.7	3351.1	0.0
Gemini (gemini-3-flash-preview)	edge_case	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	100.0
GPT-4o (gpt-4o)	edge_case	100.0	75.0	100.0	100.0	100.0	75.0	100.0	1086.7	1029.7	0.0
DeepSeek (deepseek-chat)	multi_step	100.0	66.7	100.0	100.0	100.0	66.7	33.3	5337.7	5526.2	0.0
Gemini (gemini-3-flash-preview)	multi_step	100.0	33.3	66.7	100.0	100.0	66.7	66.7	17385.0	6630.3	0.0
GPT-4o (gpt-4o)	multi_step	100.0	66.7	100.0	100.0	100.0	66.7	66.7	1053.2	1007.1	0.0
DeepSeek (deepseek-chat)	simple	100.0	100.0	100.0	100.0	100.0	100.0	100.0	3556.5	3688.8	0.0
Gemini (gemini-3-flash-preview)	simple	100.0	100.0	100.0	100.0	100.0	100.0	100.0	6988.1	10133.0	0.0
GPT-4o (gpt-4o)	simple	100.0	100.0	100.0	100.0	100.0	100.0	100.0	1529.6	1832.0	0.0
DeepSeek (deepseek-chat)	spatial	100.0	100.0	100.0	100.0	100.0	100.0	100.0	3734.3	3522.7	0.0
Gemini (gemini-3-flash-preview)	spatial	100.0	80.0	80.0	100.0	100.0	100.0	100.0	14053.8	7909.1	0.0
GPT-4o (gpt-4o)	spatial	100.0	100.0	100.0	100.0	100.0	100.0	100.0	1192.4	1325.0	0.0
DeepSeek (deepseek-chat)	synonym	100.0	60.0	100.0	100.0	100.0	60.0	0.0	4145.2	4228.0	0.0
Gemini (gemini-3-flash-preview)	synonym	100.0	60.0	100.0	100.0	100.0	60.0	0.0	9832.8	9863.6	0.0
GPT-4o (gpt-4o)	synonym	100.0	60.0	100.0	100.0	100.0	60.0	0.0	1142.3	1181.2	0.0

Architectural significance: The evaluation framework is methodology-driven rather than opportunistic. The use of ground truth labelling and field-level accuracy allows meaningful comparison beyond simple pass/fail metrics. This approach is consistent with best practices in AI system evaluation (see, for example, Liang et al., 2022, "Holistic Evaluation of Language Models"). One acknowledged limitation is that the 25-case test set may not fully represent the diversity of instructions encountered in an industrial deployment context. The absence of adversarial inputs and domain-specific terminology from ARENA2036 is a gap that Sprint 3 evaluation should address.

Task Planning Module (PB7)

The rule-based task planner receives a ParsedInstruction JSON object and a scene representation, and produces an ordered list of primitive robot commands. I designed the planner to support four action types — pick, place, move, and locate — each decomposed into a fixed sequence of lower-level operations including pre-grasp positioning, gripper actuation, transport, and release.

The planning logic is fully deterministic: given the same instruction and scene state, the planner always produces the same action sequence. This property is important for testability and debugging, as it enables reliable unit testing without mocking LLM outputs. Missing objects trigger a `ValueError` with a descriptive message identifying the missing object and listing the objects currently available in the scene, supporting clear error communication to the execution layer.

Engineering significance: The decision to use rule-based rather than LLM-based planning was deliberate. For the current constrained simulation environment — where the object set, workspace dimensions, and action vocabulary are all fixed — deterministic planning is sufficient and substantially reduces system complexity and API cost. However, this approach has known scalability limitations. The number of rules required grows with task variety, and the planner cannot generalise to instruction types not explicitly coded. Handling spatial qualifiers such as "to the left of" or "behind" is not yet supported and represents the most significant functional gap. Sprint 3 should assess whether these qualifiers can be handled within the rule-based framework or require an LLM-assisted planning step.

Execution Module and MockRobot Simulator (PB8-CMD, PB8-MOCK)

I defined the robot command schema (`action_schema.py`) as the shared interface contract between the planner and executor. This decision was made deliberately before implementation began, ensuring that all team members working on adjacent modules had a stable, documented boundary to code against. The schema defines the command structure, required fields, and permissible action types in a single location.

The MockRobot simulator maintains internal state including current end-effector position, held objects, and known scene object locations. It enforces the one-object-at-a-time constraint and validates workspace boundary conditions before executing any motion command. The executor dispatches commands sequentially, stopping and returning a structured failure summary if any command produces an error.

Robustness considerations: The MockRobot's internal state model faithfully represents the constraints of the physical simulation but does not account for non-deterministic real-world factors such as grasp failure probability, object slippage, or sensor noise. These are deliberate simplifications appropriate for the MockRobot stage. When PyBullet integration is completed in Sprint 3, the executor will need to handle asynchronous feedback from the physics engine, which introduces timing and synchronisation considerations not present in the current synchronous mock design.

End-to-End Pipeline Integration (PB10)

I connected all five pipeline stages — LLM parsing, scene lookup, task planning, execution, and feedback validation — into a single orchestrated pipeline via `main.py`. The pipeline supports both interactive and single-instruction invocation modes. Structured tracker logging is active at every

stage, recording inputs, outputs, and timing data to support debugging and future performance analysis.

The pipeline currently uses a structured stub for the vision module, which generates a fixed scene representation matching the expected schema. This stub is clearly marked in the codebase and designed for straightforward replacement once the vision module is stable.

Integration risks: The primary integration risk is schema mismatch between the vision module's real output and the stub currently used by the planner. If the vision module produces bounding box data in a different coordinate system, uses different field names, or outputs additional or missing fields, the scene-to-planner boundary will require modification. This risk should be mitigated in Sprint 3 by running an integration test with real vision outputs before the final demonstration, rather than discovering the mismatch during the demo itself.

Testing

A total of 38 unit tests were written using pytest. Tests cover all four action types in the task planner, all command types in the MockRobot, workspace boundary edge cases, object-not-found error handling, and end-to-end pipeline execution with the vision stub. All 38 tests pass without requiring API credentials or a running PyBullet environment, allowing the suite to be executed in CI and by any team member regardless of their local environment configuration.

```
(.venv) ved@Veds-MacBook-Air tests % pytest test_2.py -v
===== 38 passed in 0.02s =====
platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /Users/ved/Library/CloudStorage/OneDrive-SwinburneUniversity/Sem_5/Capstone Project/P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/ved/Library/CloudStorage/OneDrive-SwinburneUniversity/Sem_5/Capstone Project/P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning
configfile: pytest.ini
plugins: langsmith-0.7.31, anyio-4.13.0
collected 35 items

test_2.py::TestSchema::test_valid_full_instruction PASSED [ 2%]
test_2.py::TestSchema::test_valid_minimal_instruction PASSED [ 5%]
test_2.py::TestSchema::test_action_types_are_valid PASSED [ 8%]
test_2.py::TestSchema::test_confidence_levels_are_valid PASSED [ 11%]
test_2.py::TestSchema::test_invalid_action_raises PASSED [ 14%]
test_2.py::TestSchema::test_model_dump_returns_dict PASSED [ 17%]
test_2.py::TestEdgeCases::test_empty_string_is_empty PASSED [ 20%]
test_2.py::TestEdgeCases::test_whitespace_only_is_empty PASSED [ 22%]
test_2.py::TestEdgeCases::test_none_is_empty PASSED [ 25%]
test_2.py::TestEdgeCases::test_valid_instruction_not_empty PASSED [ 28%]
test_2.py::TestEdgeCases::test_vague_instruction_detected PASSED [ 31%]
test_2.py::TestEdgeCases::test_clear_instruction_not_vague PASSED [ 34%]
test_2.py::TestEdgeCases::test_colour_mention_not_vague PASSED [ 37%]
test_2.py::TestEdgeCases::test_action_mention_not_vague PASSED [ 40%]
test_2.py::TestEdgeCases::test_strips_whitespace PASSED [ 42%]
test_2.py::TestEdgeCases::test_collapses_double_spaces PASSED [ 45%]
test_2.py::TestEdgeCases::test_preserves_content PASSED [ 48%]
test_2.py::TestEdgeCases::test_vague_result_has_low_confidence PASSED [ 51%]
test_2.py::TestEdgeCases::test_vague_result_preserves_instruction PASSED [ 54%]
test_2.py::TestEdgeCases::test_vague_result_has_notes PASSED [ 57%]
test_2.py::TestEdgeCases::test_vague_result_object_is_unknown PASSED [ 60%]
test_2.py::TestEdgeCases::test_unknown_object_downgrades_confidence PASSED [ 62%]
test_2.py::TestEdgeCases::test_known_object_keeps_confidence PASSED [ 65%]
test_2.py::TestEdgeCases::test_unknown_destination_adds_notes PASSED [ 68%]
test_2.py::TestParserIntegration::test_simple_pick_instruction PASSED [ 71%]
test_2.py::TestParserIntegration::test_place_with_destination PASSED [ 74%]
test_2.py::TestParserIntegration::test_move_with_spatial_relation PASSED [ 77%]
test_2.py::TestParserIntegration::test_locate_instruction PASSED [ 80%]
test_2.py::TestParserIntegration::test_synonym_mapped_correctly PASSED [ 82%]
test_2.py::TestParserIntegration::test_ambiguous_instruction_low_confidence PASSED [ 85%]
test_2.py::TestParserIntegration::test_empty_instruction_raises PASSED [ 88%]
test_2.py::TestParserIntegration::test_whitespace_instruction_raises PASSED [ 91%]
test_2.py::TestParserIntegration::test_result_preserves_raw_instruction PASSED [ 94%]
test_2.py::TestParserIntegration::test_result_is_parsedinstruction PASSED [ 97%]
test_2.py::TestParserIntegration::test_model_dump_has_all_fields PASSED [100%]

===== 35 passed in 11.40s =====
(.venv) ved@Veds-MacBook-Air tests % []
```

```

(.venv) ved@Veds-MacBook-Air tests % pytest test_sprint2.py -v
===== test session starts =====
platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /Users/ved/Library/CloudStorage/OneDrive-SwinburneUniversity/Sem_5/Capstone Project/P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning/.venv/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/ved/Library/CloudStorage/OneDrive-SwinburneUniversity/Sem_5/Capstone Project/P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning
configfile: pytest.ini
plugins: langsmith-0.7.31, anyio-4.13.0
collected 38 items

test_sprint2.py::TestActionSchema::test_robot_command_valid PASSED [ 2%]
test_sprint2.py::TestActionSchema::test_position_as_tuple PASSED [ 5%]
test_sprint2.py::TestActionSchema::test_action_plan_total_steps PASSED [ 7%]
test_sprint2.py::TestActionSchema::test_command_summary PASSED [ 10%]
test_sprint2.py::TestActionSchema::test_plan_to_commands_simple PASSED [ 13%]
test_sprint2.py::TestActionSchema::test_plan_to_commands_with_task_id PASSED [ 15%]
test_sprint2.py::TestActionSchema::test_all_command_types_valid PASSED [ 18%]
test_sprint2.py::TestMockRobot::test_scene_loads_correctly PASSED [ 21%]
test_sprint2.py::TestMockRobot::test_move_to_valid_position PASSED [ 23%]
test_sprint2.py::TestMockRobot::test_move_to_out_of_bounds PASSED [ 26%]
test_sprint2.py::TestMockRobot::test_move_to_object PASSED [ 28%]
test_sprint2.py::TestMockRobot::test_move_to_unknown_object PASSED [ 31%]
test_sprint2.py::TestMockRobot::test_pick_success PASSED [ 34%]
test_sprint2.py::TestMockRobot::test_pick_unknown_object PASSED [ 36%]
test_sprint2.py::TestMockRobot::test_pick_when_already_holding PASSED [ 39%]
test_sprint2.py::TestMockRobot::test_place_success PASSED [ 42%]
test_sprint2.py::TestMockRobot::test_place_without_holding PASSED [ 44%]
test_sprint2.py::TestMockRobot::test_place_unknown_location PASSED [ 47%]
test_sprint2.py::TestMockRobot::test_locate_success PASSED [ 50%]
test_sprint2.py::TestMockRobot::test_locate_unknown_object PASSED [ 52%]
test_sprint2.py::TestMockRobot::test_full_pick_place_sequence PASSED [ 55%]
test_sprint2.py::TestMockRobot::test_robot_reset PASSED [ 57%]
test_sprint2.py::TestMockRobot::test_command_log_grows PASSED [ 60%]
test_sprint2.py::TestMockRobot::test_case_insensitive_object_lookup PASSED [ 63%]
test_sprint2.py::TestExecutor::test_simple_locate_plan_succeeds PASSED [ 65%]
test_sprint2.py::TestExecutor::test_full_pick_place_plan_succeeds PASSED [ 68%]
test_sprint2.py::TestExecutor::test_plan_fails_on_unknown_object PASSED [ 71%]
test_sprint2.py::TestExecutor::test_plan_stops_on_first_failure PASSED [ 73%]
test_sprint2.py::TestExecutor::test_execution_result_has_latency PASSED [ 76%]
test_sprint2.py::TestTaskPlanner::test_pick_plan_generates_steps PASSED [ 78%]
test_sprint2.py::TestTaskPlanner::test_place_plan_generates_full_sequence PASSED [ 81%]
test_sprint2.py::TestTaskPlanner::test_locate_plan_is_single_step PASSED [ 84%]
test_sprint2.py::TestTaskPlanner::test_pick_with_destination_includes_place PASSED [ 86%]
test_sprint2.py::TestTaskPlanner::test_missing_object_raises PASSED [ 89%]
test_sprint2.py::TestTaskPlanner::test_plan_preserves_instruction PASSED [ 92%]
test_sprint2.py::TestTaskPlanner::test_plan_with_task_id PASSED [ 94%]
test_sprint2.py::TestTaskPlanner::test_move_plan_same_as_place PASSED [ 97%]
test_sprint2.py::TestTaskPlanner::test_full_pipeline_planner_to_executor PASSED [100%]

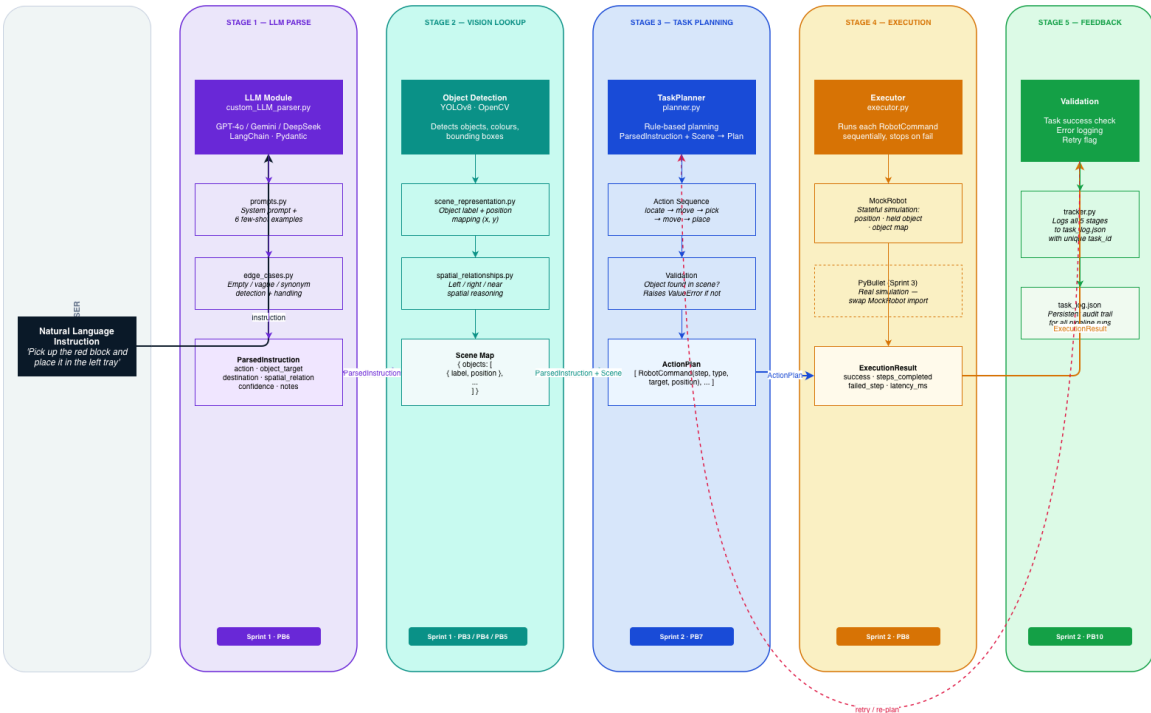
===== 38 passed in 0.02s =====

```

Limitations and future work: The current test suite is entirely unit-level. No integration tests exist that run the pipeline with real LLM outputs and real vision data simultaneously. This is the most significant validation gap in the current system and must be addressed in Sprint 3. Additionally, no performance benchmarks have been recorded for end-to-end pipeline latency under varying instruction complexity, which would be valuable for assessing system suitability for real-time robotic control.

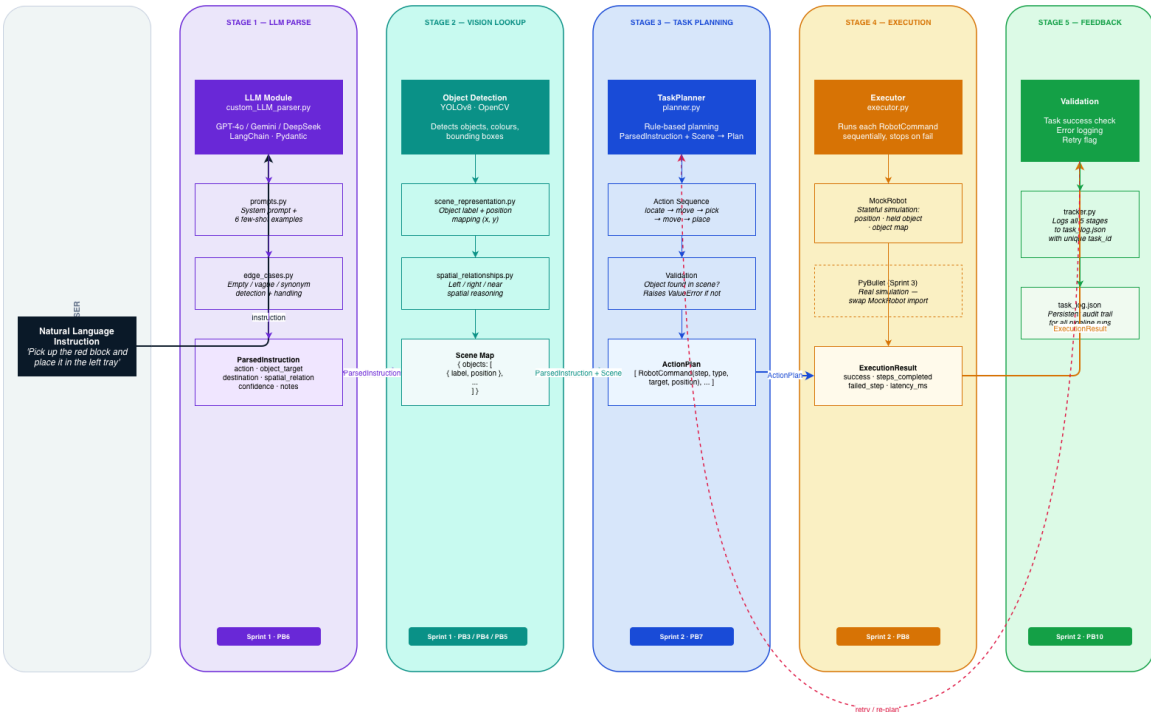
Architecture diagram showing the 5 pipeline stages and data flow

Multimodal LLM for Industrial Task Planning – System Architecture



Workflow diagram showing one instruction travelling end-to-end

Multimodal LLM for Industrial Task Planning – System Architecture



Kaveesha Dharmadasa

In the case of Sprint 2, I was the System Architect and Logic Lead, and my work focused on developing what we referred to as the "Intelligence Layer" - the part of the system that translates natural language commands into actual physical actions within the PyBullet simulation. The majority of my time was spent to ensure that the robot is capable of breaking down a high-level instruction into something safe, step-by-step, and verifiable before anything actually moves.

Key Technical Deliverables:

Standardized Task Planning Schema (PITPP-26):

I have developed a communication schema, based on JSON, to secure the interface between the LLM parsing module and the execution engine. A strict input/output contract here would have ensured that the system remained modular, with the replacement of the vision or language components later not causing a ripple effect and breaking the planning logic behind it.

Algorithmic Task Decomposition (PITPP-28):

An instruction such as Pick and Place may sound easy but when it comes to translating it into physical action, then there must be a clear sequence. I broke it down into 9 discrete primitives:

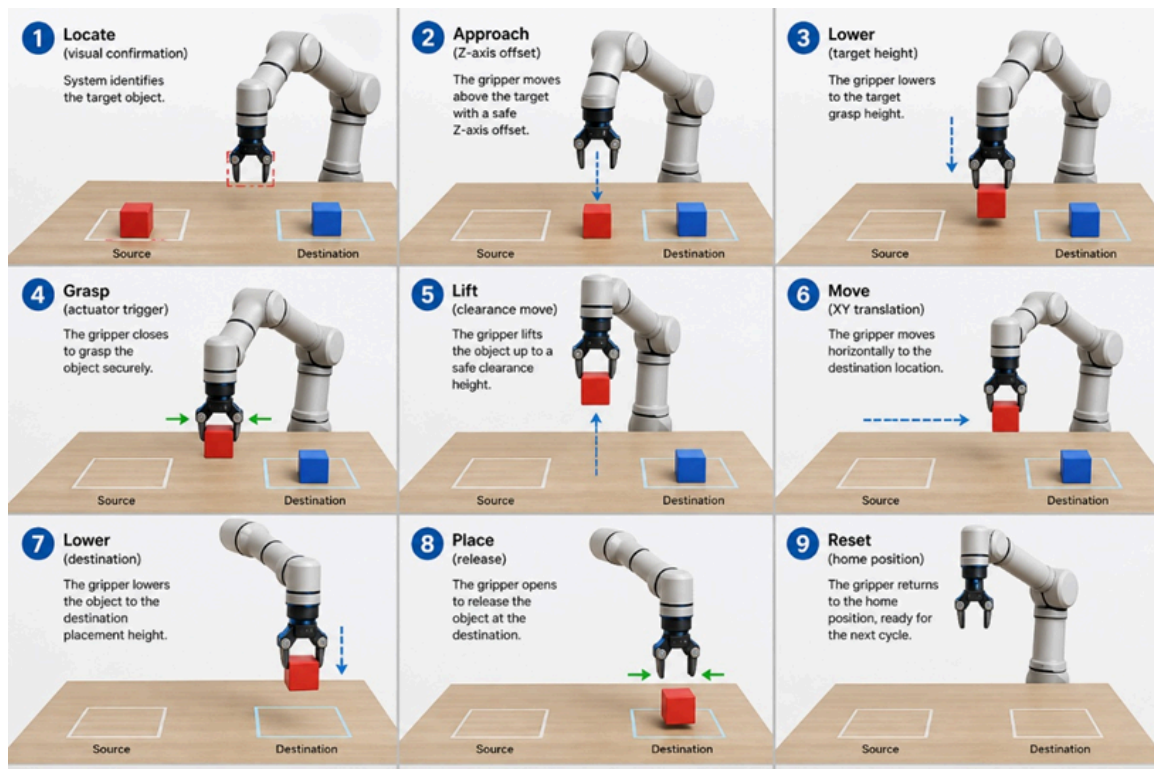
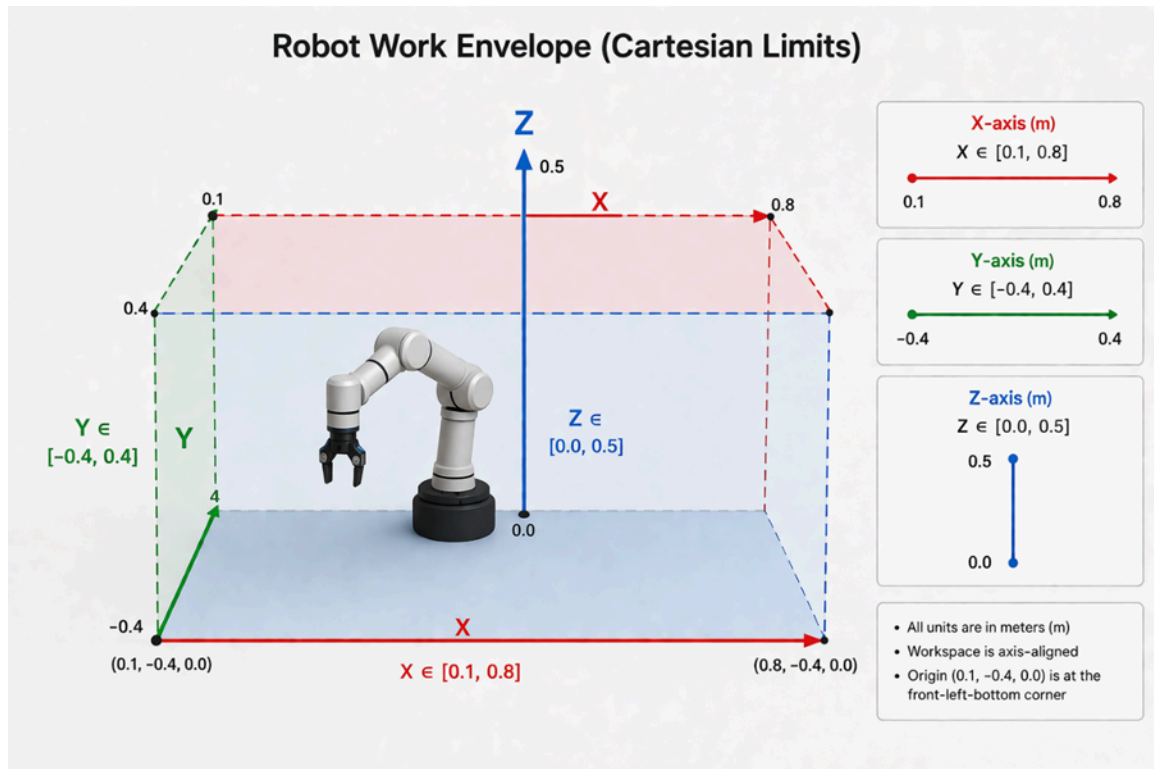


Figure 1 Visual breakdown of the multi-step task decomposition logic, illustrating the transformation of a high-level "Pick and Place" command into 9 discrete physical primitives.

Boundary Guardrails and Safety Logic Workspace Boundary Guardrails and Safety Logic (PITPP-37):

I developed a live safety interceptor that authenticated all the coordinates prior to it landing to the controller. The operational restrictions imposed are:



X-axis: $X \in [0.1, 0.8]$
 $X \in [0.1, 0.8]$
Y-axis: $Y \in [-0.4, 0.4]$
 $Y \in [-0.4, 0.4]$
Z-axis: $Z \in [0.0, 0.5]$
 $Z \in [0.0, 0.5]$

Beyond these limits, a custom BoundaryViolation exception halts the motion before it can damage hardware or cause instability in the simulation.

Quality Assurance through Test-Driven Development (PITPP-31):

I developed the testing suite in pytest with mocked environment data, which allowed me to test the decomposition logic and boundary checks of the planner in isolation - long before the Vision and LLM modules were available to integrate. Going TDD here was rewarded in final integration; most of the edge cases had already been caught and dealt with by the time final integration was reached.

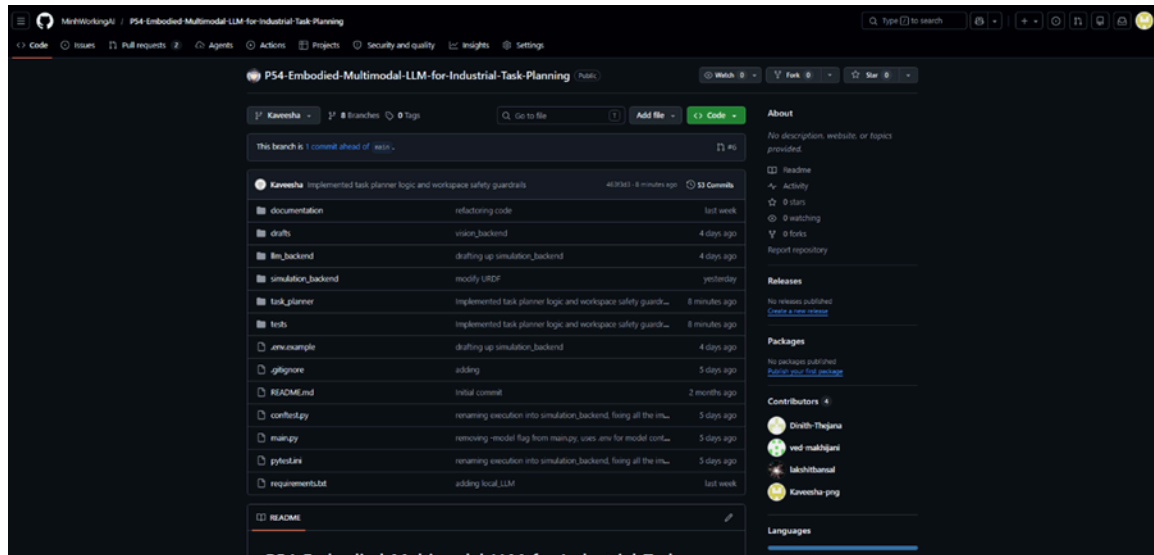


Figure 2 Overview of the 'Kaveesha' feature branch in the GitHub repository, showcasing the integrated Task Planner and Safety modules.

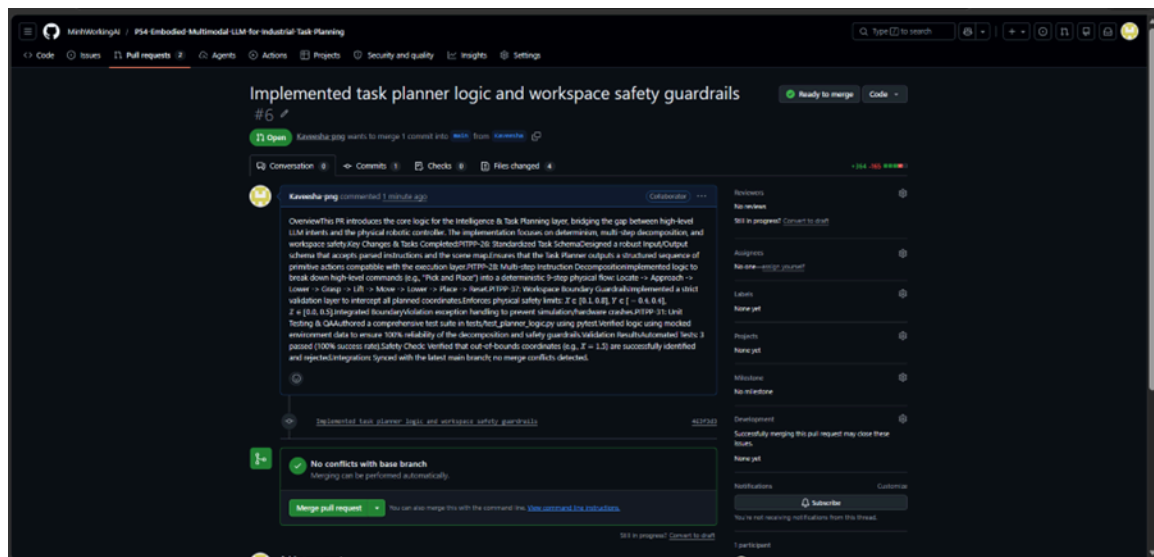


Figure 3 Detailed Pull Request documentation highlighting the implementation of task decomposition and workspace safety guardrails

```
tests > test_planner.py > ...
2
3 from llm_backend.schema import ParsedInstruction, ActionType, ConfidenceLevel
4 from simulation_backend.action_schema import Position
5 from task_planner.planner import TaskPlanner
6 from task_planner.schema import PlannerInput, PlannerOutput, PrimitiveActionType
7 from task_planner.safety import BoundaryViolation, validate_bounds
8
9
10 def _mock_scene():
11     return {
12         "objects": [
13             {"label": "red block", "position": (0.2, 0.0, 0.05)},
14             {"label": "left tray", "position": (0.6, 0.2, 0.05)},
15         ]
16     }
17
18
19 def test_pick_and_place_decomposition():
20     parsed = ParsedInstruction(
21         action=ActionType.PICK,
22         object_target="red block",
23         destination="left tray",
24         confidence=ConfidenceLevel.HIGH,
25         raw_instruction="pick and place the red block in the left tray",
26     )
27     planner_input = PlannerInput(parsed_instruction=parsed, scene=_mock_scene())
28     planner = TaskPlanner()
29
30     output = planner.generate_primitive_plan(planner_input)
31
32     assert isinstance(output, PlannerOutput)
33     action_types = {step.action for step in output.steps}
```

platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\acer\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\acer\Desktop\kaesha\PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning
configfile: pytest.ini
plugins: anyio-4.9.0, timeout-2.4.0
collected 3 items

tests/test_planner.py::test_pick_and_place_decomposition PASSED [33%]
tests/test_planner.py::test_validate_bounds_blocks_out_of_bounds PASSED [66%]
tests/test_planner.py::test_output_schema_format PASSED [100%]

===== 3 passed in 0.48s =====

PS C:\Users\acer\Desktop\kaesha\PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning>

Figure 4 Comprehensive Unit Testing Suite for Verifying Task Decomposition and Safety Constraints.

```
task_planner > safety.py > ...
1 safety.py
2 -----
3 Bounds validation helpers for the task planner.
4 ---
5
6 from simulation_backend.action_schema import Position
7
8
9 You, 24 minutes ago | I author (You)
10 class BoundaryViolation(ValueError):
11     """Raised when a target position is outside the allowed workspace bounds."""
12
13
14 def validate_bounds(position: Position) -> None:
15     """
16     Validate a position against the robot workspace bounds.
17
18     Bounds:
19     | X in [0.1, 0.8]
20     | Y in [-0.4, 0.4]
21     | Z in [0.0, 0.5]
22
23     Raises:
24     | BoundaryViolation: If any coordinate is outside the allowed range.
25     """
26     x, y, z = position.x, position.y, position.z
27     if not (0.1 <= x <= 0.8):
28         raise BoundaryViolation(f"X out of bounds: {x}")
29     if not (-0.4 <= y <= 0.4):
30         raise BoundaryViolation(f"Y out of bounds: {y}")
31     if not (0.0 <= z <= 0.5):
32         raise BoundaryViolation(f"Z out of bounds: {z}")
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
26
```

```

task_planner > schema.py > ...
11
12 from llm_backend.schema import ParsedInstruction
13 from simulation_backend.action_schema import Position
14
15
16 You, 26 minutes ago | 1 author (You)
17 class PrimitiveActionType(str, Enum):
18     APPROACH = "approach"
19     LOWER = "lower"
20     GRASP = "grasp"
21     LIFT = "lift"
22     MOVE = "move"
23     RELEASE = "release"
24     LOCATE = "locate"
25
26 You, 26 minutes ago | 1 author (You)
27 class PrimitiveAction(BaseModel):
28     action: PrimitiveActionType = Field(description="Primitive action name.")
29     target_object: Optional[str] = Field(
30         default=None,
31         description="Target object for this action, if any.",
32     )
33     target_position: Optional[Position] = Field(
34         default=None,
35         description="Target position for this action, if any.",
36     )
37     parameters: dict = Field(default_factory=dict, description="Extra parameters.")
38     description: str = Field(default="", description="Human-readable description.")
39
40 You, 26 minutes ago | 1 author (You)
41 class PlannerInput(BaseModel):
42     parsed_instruction: ParsedInstruction = Field(
43         description="Parsed LLM instruction for planning."
44     )
45     scene: dict = Field(description="Scene map from the vision module.")
46
47 You, 26 minutes ago | 1 author (You)
48 class PlannerOutput(BaseModel):
49     instruction: str = Field(description="Original instruction string.")
50     steps: list[PrimitiveAction] = Field(description="Ordered list of primitive actions.")
51     task_id: Optional[str] = Field(default=None, description="Optional tracker task ID.")
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

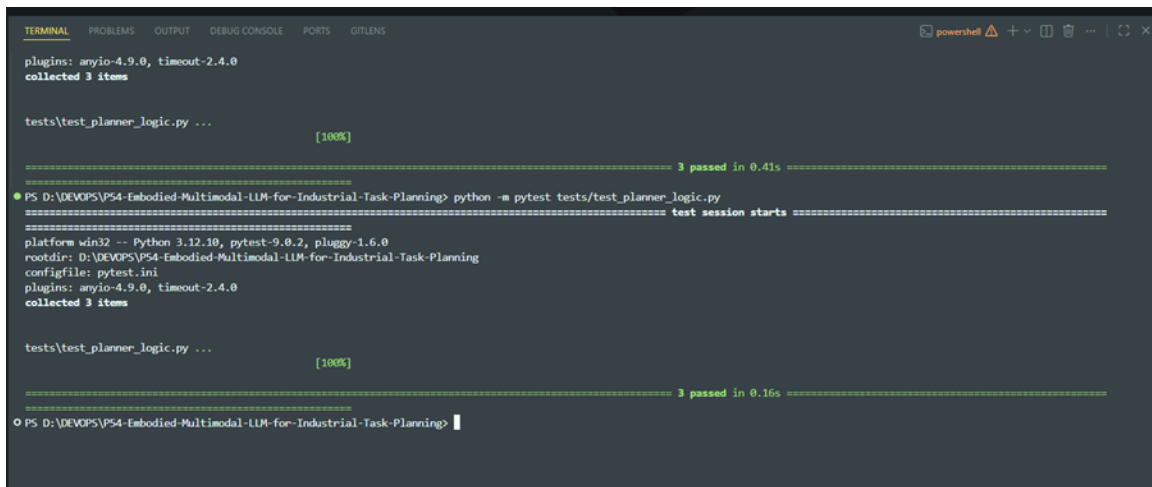
Figure 6 Implementation of Structured Input/Output Data Models and Schema Definitions in schema.py

```

39+ pass
40+
41+
42+ def validate_workspace_bounds(x: float, y: float, z: float) -> Position:
43+     """
44+     Validate that the coordinate is within safe workspace limits:
45+     X in [0.1, 0.8], Y in [-0.4, 0.4], Z in [0.0, 0.5]
46+     Raises BoundaryViolation if out of bounds.
47+     """
48+     if not (0.1 <= x <= 0.8):
49+         logger.error(f"Boundary Violation: X coordinate {x} is out of bounds [0.1, 0.8]")
50+         raise BoundaryViolation(f"X coordinate {x} out of bounds.")
51+     if not (-0.4 <= y <= 0.4):
52+         logger.error(f"Boundary Violation: Y coordinate {y} is out of bounds [-0.4, 0.4]")
53+         raise BoundaryViolation(f"Y coordinate {y} out of bounds.")
54+     if not (0.0 <= z <= 0.5):
55+         logger.error(f"Boundary Violation: Z coordinate {z} is out of bounds [0.0, 0.5]")
56+         raise BoundaryViolation(f"Z coordinate {z} out of bounds.")
57+
58+     return Position(x=x, y=y, z=z)
59+
60+
61+ # --- Scene helper ---
62+ def _find_in_scene(scene: dict, query: str) -> dict | None:
63+     """

```

Figure 7 Implementation of Workspace Boundary Guardrails in planner.py.



```

plugins: anyio-4.9.0, timeout-2.4.0
collected 3 items

tests/test_planner_logic.py ... [100%]

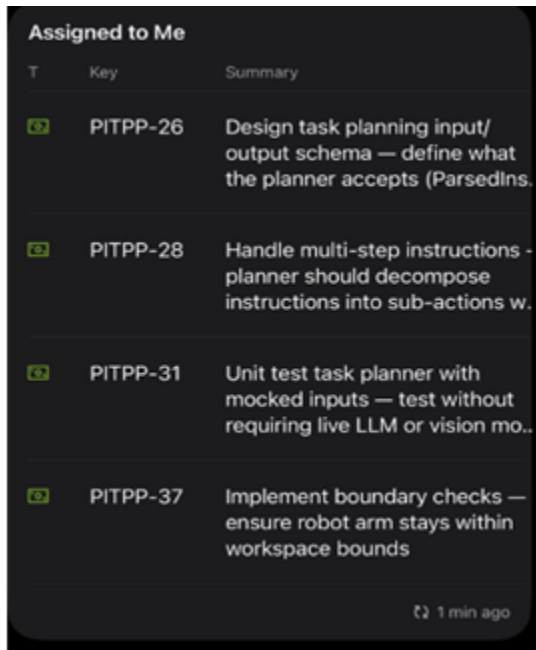
===== 3 passed in 0.41s =====
PS D:\DEVOPS\PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning> python -m pytest tests/test_planner_logic.py
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\DEVOPS\PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning
configfile: pytest.ini
plugins: anyio-4.9.0, timeout-2.4.0
collected 3 items

tests/test_planner_logic.py ... [100%]

===== 3 passed in 0.16s =====
PS D:\DEVOPS\PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning>

```

Figure 8 Automated unit testing suite for validating task decomposition.



T	Key	Summary
3	PITPP-26	Design task planning input/output schema — define what the planner accepts (ParsedIns.
3	PITPP-28	Handle multi-step instructions - planner should decompose instructions into sub-actions w.
3	PITPP-31	Unit test task planner with mocked inputs — test without requiring live LLM or vision mo..
3	PITPP-37	Implement boundary checks — ensure robot arm stays within workspace bounds

1 min ago

Figure 9 Task Board showing the assigned logic and architecture tasks.

Dinith Thejana

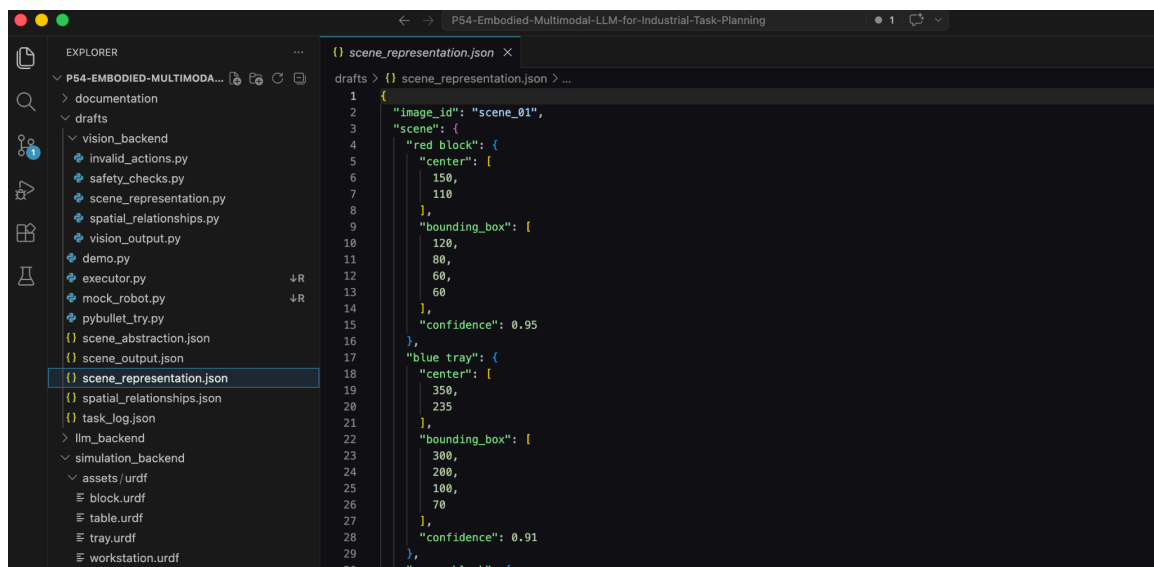
During Sprint 2, I worked on the vision integration and safety validation components of the system. My contribution mainly focused on connecting the vision module outputs with the task

planner and improving system reliability through object validation and invalid action detection. The goal of this work was to ensure that the planner could safely read scene information and prevent execution of invalid commands before actions were performed in the simulation.

Key Technical Deliverables

Scene Representation Integration (PITPP-30)

I developed the `scene_representation.py` module to transform object detection outputs into a structured JSON scene map. The implementation calculates object center coordinates from bounding box data and stores them in a planner-compatible format. This enabled the planner to read and access object positions directly from the vision module output.



```
1 {
2   "image_id": "scene_01",
3   "scene": {
4     "red block": {
5       "center": [
6         150,
7         110
8       ],
9       "bounding_box": [
10        120,
11        80,
12        60,
13        60
14      ],
15      "confidence": 0.95
16    },
17    "blue tray": {
18      "center": [
19        350,
20        235
21      ],
22      "bounding_box": [
23        300,
24        200,
25        100,
26        70
27      ],
28      "confidence": 0.91
29    },
30  },
31 }
```

Figure 1: Scene representation JSON output

Safety Validation Checks (PITPP-36)

I implemented pre-execution safety validation to ensure that commands are only executed for objects available within the scene. Testing was conducted using `test_safety_check.py`, where existing objects were validated successfully while invalid objects generated appropriate error messages.

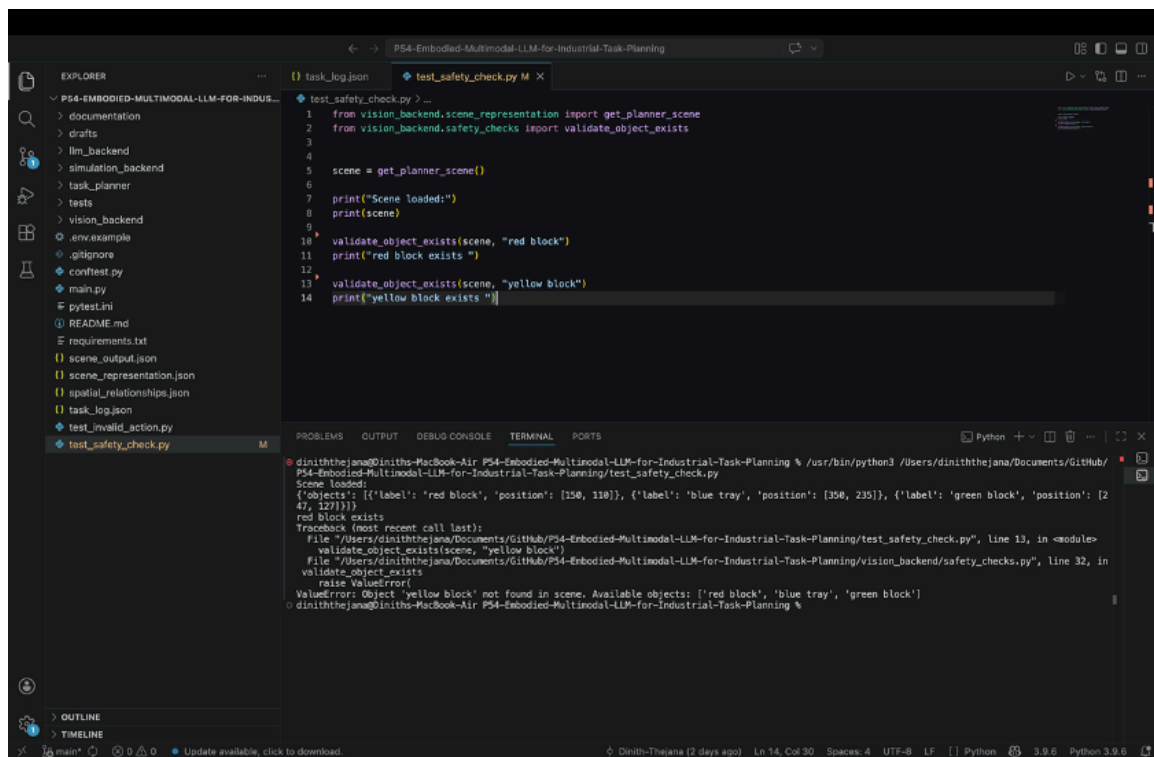


Figure 2: Safety validation test showing successful and failed object check

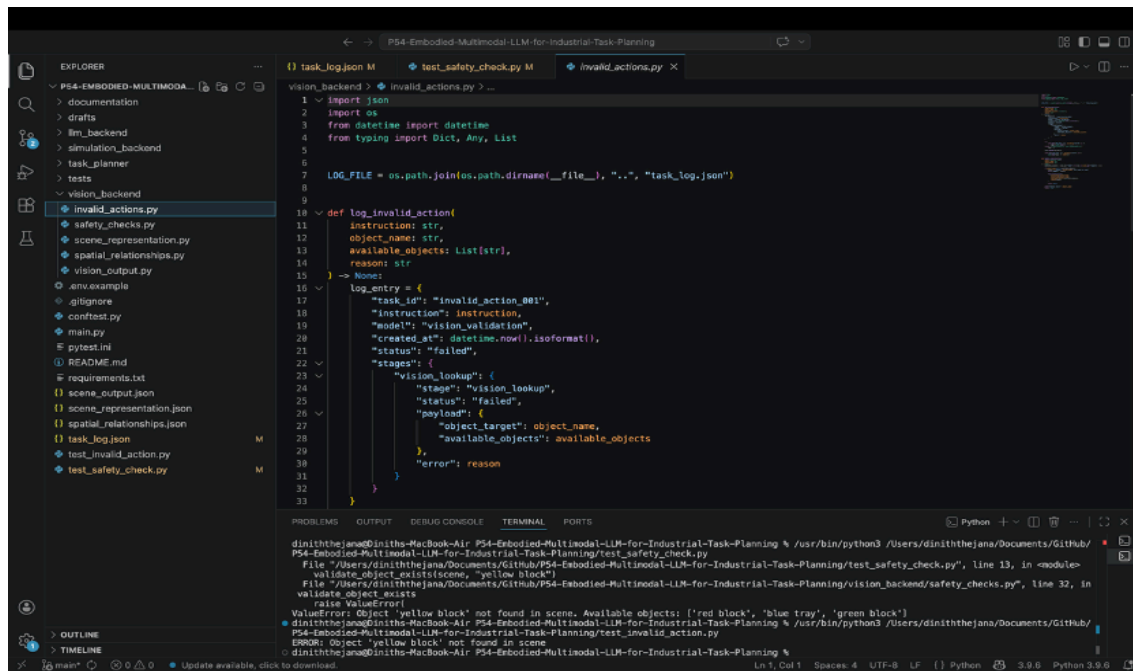
Invalid Action Detection and Logging (PITPP-38)

I developed the `invalid_actions.py` module to detect invalid commands such as attempting to interact with objects that do not exist in the environment. Failed actions were recorded in `task_log.json` using structured JSON logging, improving debugging, monitoring, and execution traceability.

The module was designed to validate planner instructions before execution by comparing requested objects against the available scene objects generated by the vision module. If an object was missing from the scene, the system generated an appropriate error message and prevented the command from continuing to execution. This reduced the possibility of unsafe or incorrect robot actions within the simulation environment.

I also implemented detailed JSON logging to store information such as task ID, instruction, validation stage, failure status, available objects, timestamps, and error messages. This logging mechanism provided better transparency for debugging and allowed failed actions to be tracked systematically during testing and development.

Testing was performed using `test_invalid_action.py`, where invalid instructions such as attempting to pick up a non-existent “yellow block” were successfully detected and logged. Terminal outputs and generated JSON records confirmed that the system could correctly identify invalid actions and provide meaningful feedback for troubleshooting and future improvements.



The screenshot displays the Visual Studio Code editor with the file `invalid_actions.py` open. The file contains a function `log_invalid_action` that logs an invalid action. The terminal output shows the execution of `test_invalid_action.py`, which calls `log_invalid_action` with an invalid instruction. The output includes a JSON log entry for the invalid action and a `ValueError` message indicating that the object 'yellow block' was not found in the scene.

```
1 import json
2 import os
3 from datetime import datetime
4 from typing import Dict, Any, List
5
6 LOG_FILE = os.path.join(os.path.dirname(__file__), "..", "task_log.json")
7
8
9
10
11 def log_invalid_action(
12     instruction: str,
13     object_name: str,
14     available_objects: List[str],
15     reason: str
16 ) -> None:
17     log_entry = {
18         "task_id": "invalid_action_001",
19         "instruction": instruction,
20         "model": "vision_validation",
21         "created_at": datetime.now().isoformat(),
22         "status": "failed",
23         "stages": {
24             "vision_lookup": {
25                 "stage": "vision_lookup",
26                 "status": "failed",
27                 "payload": {
28                     "object_target": object_name,
29                     "available_objects": available_objects
30                 },
31                 "error": reason
32             }
33         }
34     }
```

```
dinithejane@dinithe-MacBook-Air: PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning % /usr/bin/python3 /Users/dinithejane/Documents/Github/PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning/test_invalid_action.py
File "/Users/dinithejane/Documents/Github/PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning/test_invalid_action.py", line 13, in <module>
    validate_object_exists(scene, "yellow block")
File "/Users/dinithejane/Documents/Github/PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning/vision_backend/safety_checks.py", line 32, in
    validate_object_exists
    raise ValueError(
ValueError: Object 'yellow block' not found in scene. Available objects: ['red block', 'blue tray', 'green block']
PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning %
```

Figure 3: Invalid action detection terminal output

Testing and Evidence

I tested all modules locally using Python scripts and terminal execution within VS Code. Multiple JSON outputs, including `scene_representation.json` and `task_log.json`, were generated successfully as evidence of planner integration, safety validation, and invalid action handling.

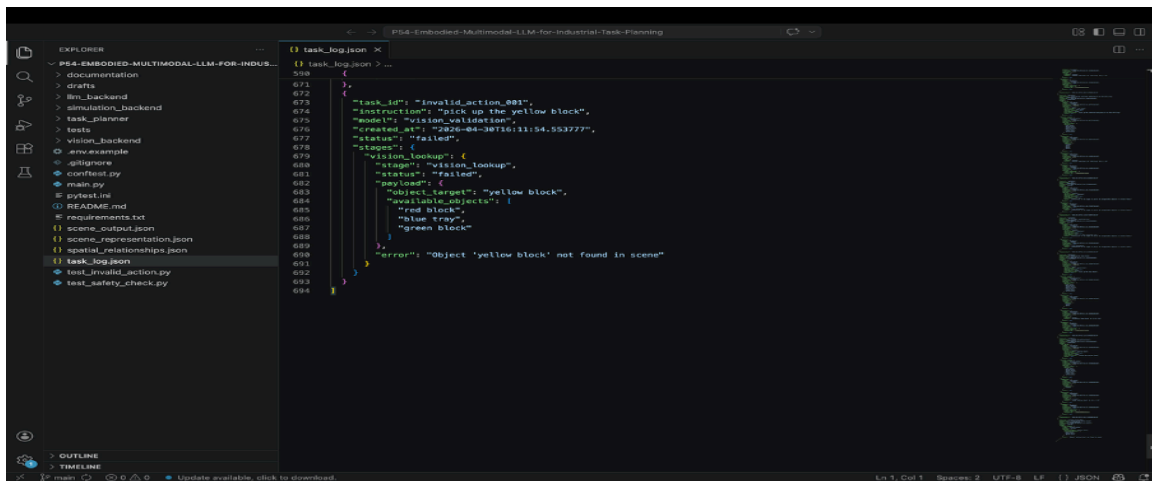


Figure 4: Task log JSON showing failed action recording

Lakshit Bansal

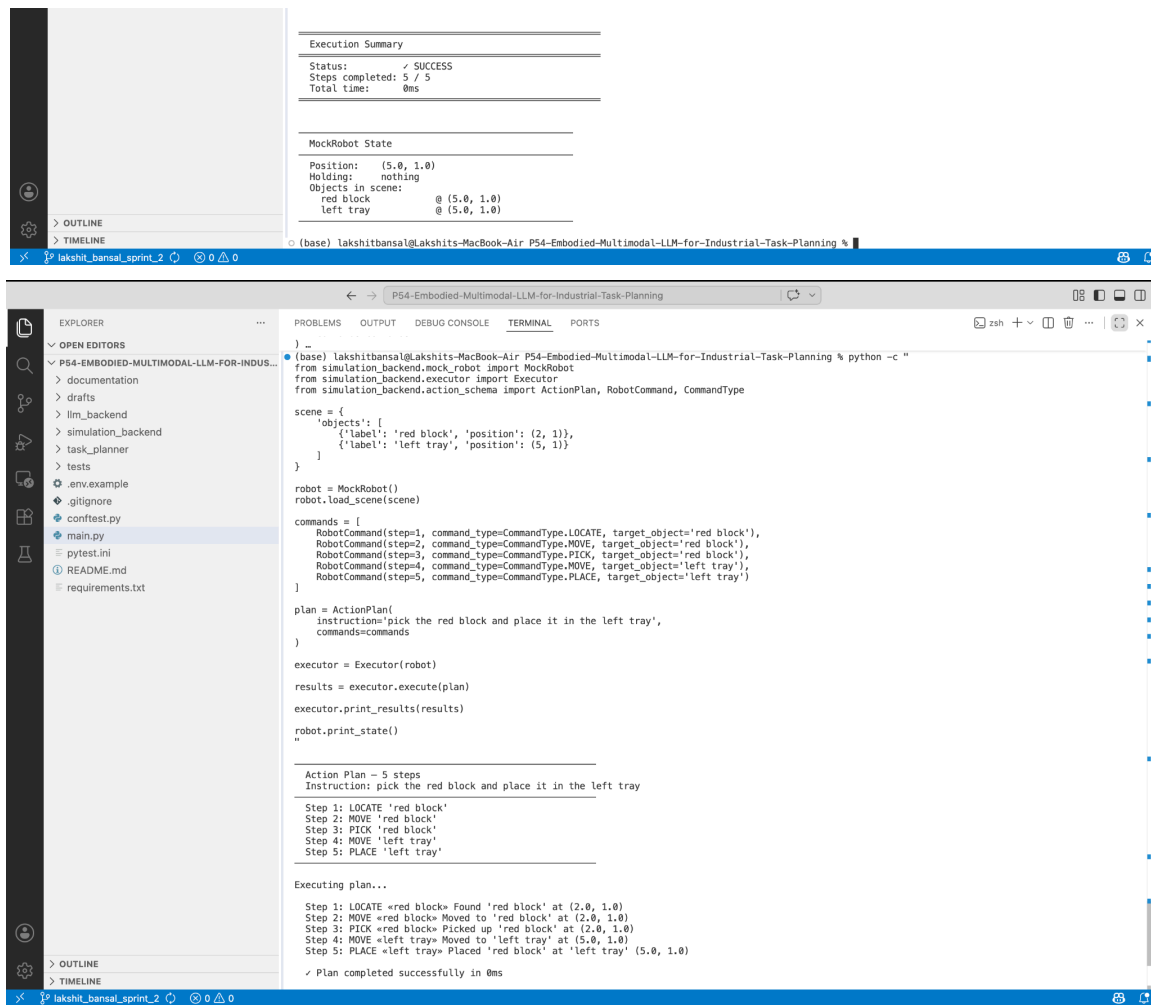
During Sprint 2, my primary contribution focused on the execution and integration layers of the embodied industrial task planning pipeline. My work concentrated on bridging the gap between high-level task planning outputs and executable robot behaviour within the simulation environment. This involved implementing the execution backend, improving execution reliability, and contributing to full end-to-end pipeline integration and testing.

Key Technical Deliverables:

Execution Backend Development (PITPP-33, PITPP-35)

I implemented and tested the execution backend using the Executor and MockRobot modules. The execution module translates planner-generated commands into robot actions such as locate, move, pick, and place. Sequential execution handling was implemented to ensure that commands execute in the correct order and stop safely if an error occurs during execution.

From a system architecture perspective, the execution backend acts as the bridge between high-level reasoning and low-level robot behaviour. This separation was intentionally designed to keep the planning logic independent from the underlying simulation environment, allowing the same planner outputs to later be reused with a real PyBullet robot or physical hardware without major architectural changes.



The MockRobot environment was used to simulate robot behaviour without requiring full PyBullet integration. The robot maintains internal state including current position, held objects, and scene object locations, allowing the execution pipeline to be tested independently before connecting to the real simulation backend.

Figure 1: Successful sequential execution of a full pick-and-place action sequence using the Executor and MockRobot modules.

Execution Timeout and Failure Handling (PITPP-39)

I added timeout handling and graceful failure logic to improve execution reliability. The executor monitors execution duration for each command and flags tasks that exceed the configured timeout threshold. This improves system robustness and prevents the pipeline from hanging during execution failures or invalid commands.

Additionally, the execution module was designed to stop on failure and return structured execution summaries containing completed steps, failure messages, and latency metrics.

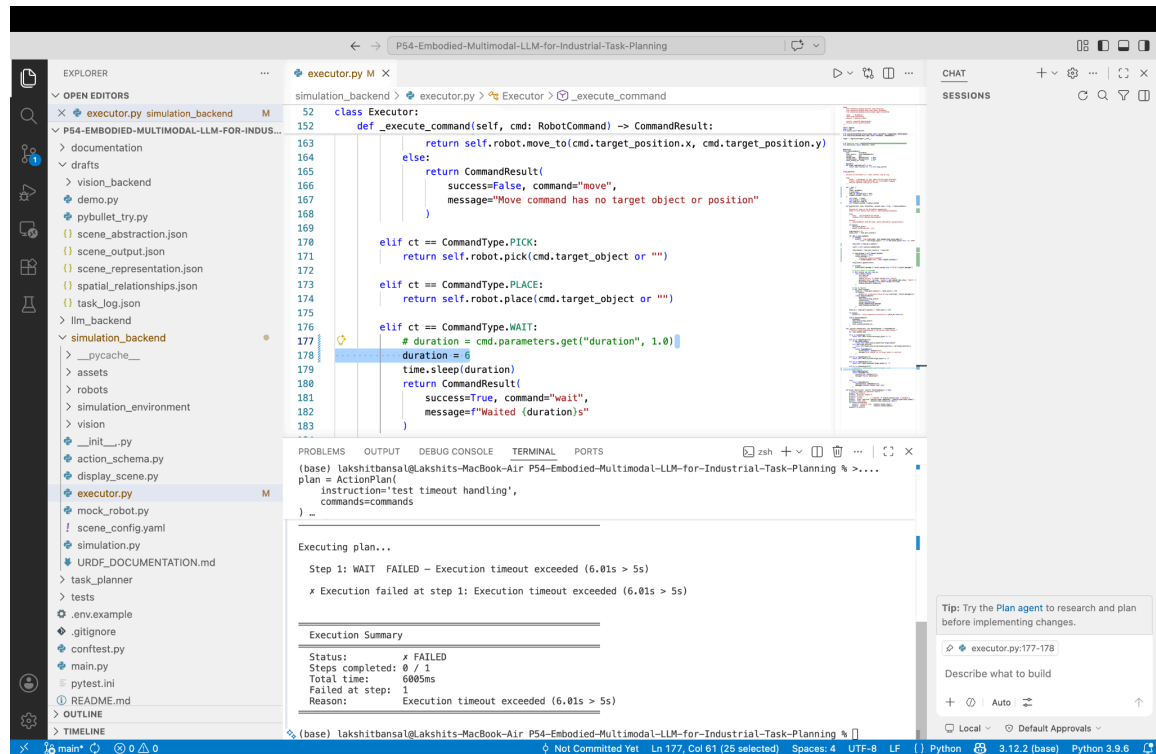


Figure 2 Execution summary generated after successful pipeline execution and validation.

One limitation of the current execution backend is that all execution behaviour is still synchronous and operates entirely within the MockRobot environment. Real-world robotics environments introduce additional complexity such as asynchronous sensor feedback, grasp uncertainty, physics instability, and motion timing constraints that are not yet represented in the current implementation. While the current execution layer is sufficient for validating pipeline integration and command sequencing, Sprint 3 will require adaptation of the executor for real PyBullet feedback handling and more advanced recovery behaviour.

Minh Hoang Duong

Folder Restructure, Import Architecture, Codebase Refactoring and Maintenance

During Sprint 2, my contributions spanned codebase architecture, simulation backend design, and system maintenance. I acted as the de facto technical lead for infrastructure, refactoring, and simulation environment setup — ensuring the codebase remained clean, maintainable, and structurally sound as the team built across multiple parallel workstreams.

Throughout the sprint I proactively maintained code quality, resolving multiple critical bugs and structural issues that were blocking pipeline execution. Key fixes included a broken relative import in `llm_backend/backends/__init__.py` where the HuggingFace backend used an absolute import path (`from backends.huggingface_backend`) instead of the correct relative import (`from .huggingface_backend`), causing an immediate crash on startup.

I restructured the entire codebase from a flat layout — where all files lived at the project root — into a properly organised package hierarchy. The key changes were moving evaluation files into `llm_backend/LLM_eval/`, moving tests into `tests/`, renaming `experiments/` to `draft/`, and renaming `execution/` to `simulation_backend/` in preparation for real PyBullet integration.

A `conftest.py` was created at the project root to add the project directory to `sys.path` automatically for all pytest runs, and `pytest.ini` was created to register the custom `integration` mark and suppress false collection of `test_cases.py`.

The reason behind each folder's location is deliberate. `llm_backend/backends/` isolates all LLM client construction so no other module needs to know about API keys or LangChain internals. `llm_backend/LLM_eval/` separates the evaluation workflow from the production pipeline — they share `schema.py` and `edge_cases.py` but are otherwise independent.

Simulation Backend Architecture and Design

`simulation_backend/` encapsulates everything physics-related so the rest of the pipeline never imports PyBullet directly.

`simulation_environment/` contains files responsible for building the physical world — `workspace.py` builds the table surface, floor plane, and four perimeter walls; `object_loader.py` reads the `objects` list from `scene_config.yaml` and loads each URDF into PyBullet, applying colours at runtime via `changeVisualShape()`; `object_registry.py` maintains a bidirectional map between PyBullet integer body IDs and human-readable object labels, enabling the vision module to resolve segmentation mask pixel values back to object names; `scene_builder.py` assembles the rich scene dictionary from detection results, computes spatial relationships (`on_top_of`, `near_to`) from 3D positions, and provides a `to_planner_format()` adapter so `planner.py` receives the format it already expects without modification.

`robots/` contains `RobotBase` — an abstract class enforcing the interface contract (`move_to`, `pick`, `place`, `locate`, `load_scene`, `reset`) — with concrete implementations for Franka Panda, UR5, and Kuka IIWA. Gripper classes (`GripperBase`, `FrankaHand`, `Robotiq85`) live in `robots/gripper/`. Robot selection is controlled by `SIMULATION_ROBOT=Franka/Franka_Panda` in `.env`, following the same manufacturer/model naming convention as HuggingFace model identifiers.

`vision/` contains `DetectorBase` — an abstract interface all CV detectors implement — with `detection_implementation/` housing concrete detector files (`yolo_detector.py`, `rtdetr_detector.py`, `grounding_dino_detector.py`, `sam_detector.py`).

Detector selection is controlled by `VISION_DETECTOR=yolov8n` in `.env`.

`SEGMENTATION_MASK=true/false` controls an independent segmentation channel that runs alongside the primary detector for academic comparison and fallback purposes. Model weights are stored flat in `vision/detection_models/` (gitignored, downloaded on first run).

`scene_config.yaml` was designed as the single configuration source for the entire simulation — physics parameters, workspace dimensions, camera position, display settings, and the full object list with URDF paths, colours, positions, masses, and graspability flags. Adding a new object to the scene requires only a new YAML entry — no code changes.

Ground Truth and Factory Design

`vision/ground_truth.py` implements the last-resort scene source. Rather than using a camera or CV model, it calls `p.getBasePositionAndOrientation(body_id)` for every object registered in `ObjectRegistry`, returning exact positions directly from the PyBullet physics state. Detections are returned as `Detection` dataclass instances with `confidence=1.0` and `source="ground_truth"`. This is never the primary source — it only activates when both the primary detector and segmentation mask fail to find an object, at which point a warning is logged.

The factory pattern appears three times in the simulation backend, mirroring the LLM backend:

- `robots/__init__.py` — `get_robot()` reads `SIMULATION_ROBOT` from `.env` and routes to the correct robot file
- `vision/__init__.py` — `get_detector()` reads `VISION_DETECTOR` from `.env` and routes to the correct detector file
- `llm_backend/backends/__init__.py` — `get_llm()` reads `LLM_BACKEND` from `.env`

`display_scene.py` Implementation

I implemented `display_scene.py` as a standalone visual debugging tool runnable only via `python -m simulation_backend.display_scene`. It is intentionally isolated — it cannot be imported or called from anywhere in the pipeline.

The tool opens two windows simultaneously: a PyBullet GUI window showing the full 3D physics scene with floating object labels rendered via `p.addUserDebugText()` and a robot state overlay via `p.addUserDebugItem()`, and an OpenCV window showing the live camera feed from the configured eye-to-hand camera, with optional segmentation mask overlay when `SEGMENTATION_MASK=true`.

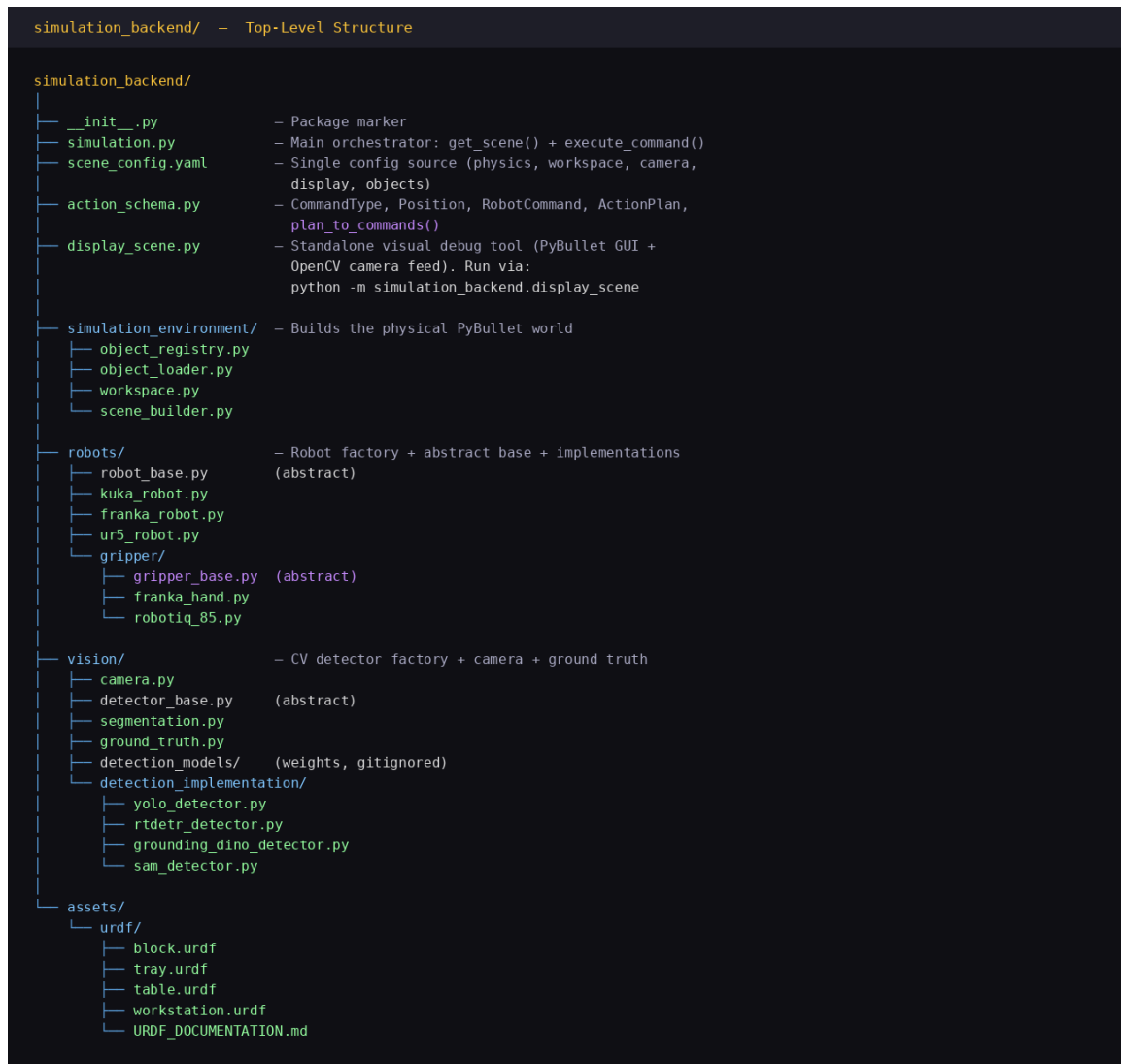


Figure 1: `simulation_backend` tree graph of files organization (Generated by Claude Sonnet 4.6) “display the folder structure and the files function in tree graph of simulation backend. In

markdown format, convert it into image downloadable jpg or png files, but keep the structure format and display of the markdown ones that are generated.”

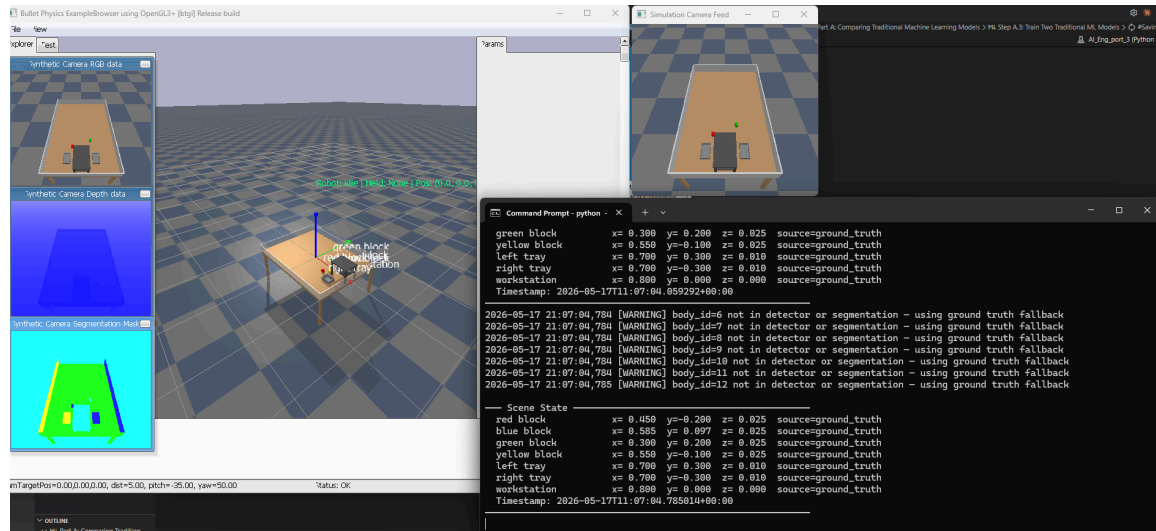


Figure 2: `display_scene.py` implementation running

1. SPRINT PLAN

Sprint 2 (Weeks 7–9) was structured around four epics that directly extend the deliverables from Sprint 1. Sprint 1 produced a functional LLM instruction parser with a stable output schema (ParsedInstruction). Sprint 2 built the downstream pipeline that consumes that output and translates it into executable robot behaviour.

Epic 1 — Comparative Evaluation (EVAL-1, EVAL-2, EVAL-3): Execute the multi-model evaluation framework developed at the end of Sprint 1, analyse results, and present findings to the client. This was prioritised early in the sprint as it directly addressed client feedback from the Sprint 1 review.

Epic 2 — Task Planning Module (PB7): Design and implement a rule-based task planner that takes ParsedInstruction JSON and a scene representation as input and produces an ordered action plan. A rule-based approach was selected over an LLM-based planner for Sprint 2 on the grounds of determinism, testability without API calls, and suitability for the constrained simulation environment.

Epic 3 — Execution Module (PB8): Define a shared robot command schema (PB8-CMD) as an interface contract, then implement a MockRobot simulator (PB8-MOCK) that responds to commands with real internal state changes. This design

allowed the full pipeline to be developed and validated independently of the PyBullet physics engine.

Epic 4 — System Integration (PB10): Connect all modules into a single end-to-end pipeline via `main.py`, with cross-domain logging at every stage. Vision module integration uses a clearly marked stub, pending completion of the vision sub-system by teammates.

The three sprints form a deliberate progression: Sprint 1 delivered the LLM module and evaluation framework; Sprint 2 added the planning and execution layers; Sprint 3 will complete real simulation integration, end-to-end validation with live scene data, and the final demonstration.

2. SPRINT PROGRESS

Ved's progress :

EVAL-1 and EVAL-2 were completed in Week 7. The `comparison_report.py` evaluation script was executed across all 25 test cases for GPT-4o, Gemini, and DeepSeek. Results were exported to `evaluation_metrics.csv` and `evaluation_results.json` for analysis. GPT-4o achieved the highest instruction accuracy overall, performing particularly well in the spatial and multi-step categories. DeepSeek demonstrated competitive performance in the simple and synonym categories at lower latency. Gemini exhibited the greatest variance across instruction categories, suggesting sensitivity to phrasing differences. A full metric breakdown — covering parse success rate, instruction accuracy, field-level accuracy, latency, and error rate — was computed per model and per category.

[Table: Model Evaluation Results — GPT-4o / Gemini / DeepSeek across 25 test cases. Columns: Model, Parse Success Rate (%), Instruction Accuracy (%), Field-Level Accuracy (%), Mean Latency (ms), Error Rate (%). To be populated from evaluation_metrics.csv.]

The task planning module (PB7) was completed and unit tested. The planner correctly generates action sequences for all four supported action types. Attempts to plan around unavailable objects raise a `ValueError` with a descriptive message listing the objects currently present in the scene.

The robot command schema (PB8-CMD) and mock simulator (PB8-MOCK) were both completed. The `MockRobot` maintains internal state across a full pick-and-place sequence, enforces a one-object-at-a-time constraint, validates workspace boundary conditions, and handles unknown object references gracefully.

The pipeline scaffold (PB10, `main.py`) was completed with all five stages connected and tracker logging active at each stage. The pipeline supports both interactive use and single-instruction invocation via command-line argument, with model selection via `--model` flag.

A total of 38 unit tests were written using pytest and all passed without requiring API credentials or a PyBullet environment.

Minh Hoang Duong Progress:

During Sprint 2, I led the simulation backend architecture and codebase infrastructure workstream, operating as the technical lead for system maintenance and simulation environment design.

I conducted a comprehensive audit of the codebase, identifying and resolving multiple critical defects that were preventing pipeline execution. This included correcting import resolution errors, eliminating redundant configuration flags, and standardising initialisation patterns across all backend modules. All changes were documented through descriptive version control commits.

I performed a full structural reorganisation of the project, migrating from a flat file layout to a properly layered package hierarchy. All affected import paths were resolved, and a standardised test infrastructure was established.

I refactored the model registry module to eliminate duplicated construction logic and enforce a single source of truth for LLM client instantiation, consistent with the architectural patterns established across the rest of the system.

I designed and implemented the full simulation backend architecture, applying the factory design pattern established in the LLM module to support three swappable industrial robot configurations and five swappable computer vision detector models, each selectable via environment variable without code modification.

Kaveesha Dharmadasa Progress

PITPP-26, PITPP-28, PITPP-31, and PITPP-37 were completed during Sprint 2. The main focus of this sprint was the development of the Task Planning and Safety Layer for the multimodal robotic system. A structured planner was implemented to transform high-level natural language instructions into executable primitive robot actions within the PyBullet simulation environment .

The task planning module was designed using a deterministic rule-based approach for Sprint 2 to ensure predictable and testable behaviour before introducing more advanced LLM-driven planning in future sprints. The planner successfully converts instructions such as “pick and place the red block” into structured action sequences including approach, lower, grasp, lift, move, and release primitives .

A standardized communication schema was also implemented using Pydantic models to establish a stable interface between the LLM parser, vision module, and task planner. This improved modularity and ensured consistent data exchange across system components .

Workspace safety guardrails were implemented through the safety validation module. Boundary checks were added for X, Y, and Z coordinates to prevent unsafe robot movements during execution. Invalid coordinates automatically trigger a custom `BoundaryViolation` exception before commands reach the controller

Team progress:

Lakshit Bansal and Kaveesha Dharmadasa completed the vision framework research and YOLOv8 object detection prototype. Dinith Thejana implemented `scene_representation.py` and `spatial_relationships.py`.

3. SPRINT REVIEW

1. What was demonstrated to the client:

The Sprint 2 review was conducted in the week between week 9 and 10 . The team demonstrated the end-to-end pipeline operating on a pick-and-place instruction, showing instruction parsing via the Gemini backend, rule-based task plan generation, and sequential execution through the MockRobot simulator. The comparative evaluation results across GPT-4o, Gemini, and DeepSeek were also presented, including per-category accuracy and latency findings.

The implementation of `simulation_backend/` with the code structure, folder structure design along with visual display via `display_scene.py` was also demonstrated.

2. Client feedback:

The Sprint 2 increment was demonstrated to the client and received positively, with the client formally accepting it as a deliverable. The client noted with encouragement the visible progress in system integration, acknowledging the collective effort as the individual workstreams began converging into a cohesive pipeline.

The clients also provided constructive written feedback following the sprint review, identifying several areas for improvement in the sprint report. The key observations are summarised below.

Several sections remained incomplete at the time of submission, most notably the Sprint Review and Client Feedback sections, which contained placeholder text.

The report title and task numbering also reflected inconsistencies that would benefit from revision.

Sentence structure, grammar, and paragraph composition were noted as areas requiring greater care, with some passages identified as overly lengthy or awkwardly phrased.

The level of contribution detail varied considerably across team members. Some sections were technically thorough and well-evidenced, while others lacked comparable depth and supporting evidence. A more balanced and unified writing style across all sections would strengthen the report's academic presentation.

The report was noted as heavily implementation-focused, with extensive references to file names, module outputs, and test scripts. While this demonstrates technical progress, the feedback indicated a need for greater emphasis on engineering significance — explaining why architectural decisions were made, what trade-offs were considered, and how the system design addresses the project's broader goals.

On the evaluation side, while the 38-test result was noted, the feedback indicated that deeper discussion of scalability, robustness, integration risks, and AI system limitations would strengthen the critical analysis. Validation results and failure-case discussion in particular were identified as requiring further elaboration.

Finally, academic referencing and engagement with relevant literature were noted as limited throughout the report. Grounding technical decisions in published research — particularly around LLM evaluation methodology, robot task planning, and simulation-based validation — would improve the report's academic rigour and research contribution.

3. Critical analysis:

Sprint 2 was more successful than Sprint 1 in terms of scope completion. All four epics were addressed and the core pipeline is demonstrable end-to-end. The decision to use a MockRobot and vision stub proved to be the correct engineering call — it eliminated cross-team dependencies during development while keeping integration points explicit and well-defined.

However, several limitations warrant acknowledgement. The rule-based task planner is well-suited to the current constrained simulation environment but lacks the flexibility to handle novel or ambiguous spatial instructions without explicit rule additions. This represents a scalability concern: as task complexity increases, the number of required rules grows non-linearly and the planner becomes increasingly brittle. Sprint 3 should evaluate whether a hybrid approach — rule-based planning with LLM-assisted disambiguation — is warranted.

The 38-unit test suite provides a strong quality baseline, but all tests operate against the MockRobot and stub scene data. Integration testing — where the full pipeline runs with real vision module outputs and live PyBullet simulation — has not yet been performed. This represents a significant validation gap and constitutes the primary technical risk entering Sprint 3. If the vision module outputs differ from the stub schema in structure or field naming, pipeline failures will occur at the scene-to-planner boundary

From a robustness perspective, the current pipeline does not handle partial execution failure gracefully beyond logging and halting. A more resilient execution model — for example, one that supports rollback or replanning on failure — should be considered as the system moves toward real hardware scenarios.

The comparative LLM evaluation, while rigorous within its 25-test scope, is limited in generalisability. The test set does not include adversarial inputs, linguistically complex instructions, or domain-specific terminology from the ARENA2036 industrial context. Expanding the evaluation set in Sprint 3 would strengthen the research contribution and provide more credible benchmark results.

AI limitations that remain unaddressed include: the absence of a confidence score or uncertainty measure in LLM parser outputs, the lack of fallback behaviour when the LLM produces an output that fails schema validation, and no mechanism for handling ambiguous instructions that could map to multiple valid action plans. These are areas of known risk for production deployment and should be addressed or at minimum documented in the final report.

4. RETROSPECT (CRITICAL REVIEW OF THE PROCESS)

Team plan: The sprint plan was well-structured with clear epic ownership. Parallel development across independent workstreams — Ved on planning and execution, Lakshit and Kaveesha on the task planner internals, Dinith on scene representation, Minh on simulation setup — avoided blocking during development but created an integration risk at the sprint boundary. Sprint 3 should include an explicit integration milestone in Week 10 rather than treating integration as an emergent outcome.

Communication plan: Internal communication via Microsoft Teams was effective overall. However, daily standups were not consistently held, and some blockers — particularly the PyBullet macOS compatibility issue — were not escalated early enough to be resolved within the sprint. An async daily update in the Teams channel should be enforced in Sprint 3 to improve visibility of blockers.

Quality plan: The 38-test pytest suite established a strong unit-level quality baseline. Integration tests — running the full pipeline with real module outputs — were not

achievable in Sprint 2 due to the vision stub. Sprint 3 must include integration testing as a formal quality gate before the final demonstration.

Risk registry: The primary risk that materialised this sprint was PyBullet environment incompatibility across team members' machines, which blocked real simulation execution. The MockRobot mitigation was effective for development continuity, but the underlying environment risk remains unresolved. A shared Docker container or university computing environment should be established in Sprint 3. A secondary risk is dependency on external LLM APIs for full pipeline testing; this was partially mitigated by the unit test suite's ability to run without API calls, but live integration testing still requires API credits.

Ethical considerations: The multi-model evaluation sends user instructions to external API endpoints operated by third parties (OpenAI, Google, DeepSeek). For an industrial deployment context, sensitive task instructions should be processed locally. This aligns with the client's earlier suggestion to explore a local LLM fallback option in Sprint 3. Additionally, the workspace boundary validation and one-object-at-a-time constraint built into the MockRobot reflect responsible design practice for physical automation systems, where unvalidated motion commands carry real safety consequences.

5. LESSONS LEARNED (CRITICAL REVIEW OF SPRINT ONE EXPERIENCE AND FUTURE PLAN)

- **Define interface contracts before implementation.** Writing `action_schema.py` before building the executor or planner eliminated integration conflicts entirely. This should be the first step for every new module in Sprint 3.
- **Stub-based development is a legitimate engineering strategy.** Using a clearly marked vision stub in `main.py` allowed full pipeline development to proceed without cross-team dependency. The integration point remained explicit and easy to replace.
- **Rule-based planning is effective for constrained environments, but has limits.** The task planner is fully deterministic and testable without an API. For the current simulation scope it is sufficient; however, Sprint 3 should evaluate its behaviour on more complex spatial instructions before concluding it is adequate for the final demonstration.
- **Evaluation rigour strengthens research credibility.** The structured evaluation — six categories, 25 test cases, per-category ground truth, and five metrics — produced findings that were well-received by the client. The same level of rigour should be applied to full pipeline evaluation in Sprint 3.
- **Sprint 3 priorities:** Begin vision integration on the first day of Week 10. Establish a shared development environment. Add spatial relation handling to the task planner. Run full end-to-end integration tests with real scene data before the final demonstration.

